

↳ tree that represents the geometrical structure

↳ Entree of crests infants needs no crests just what we
can for new CLASS

By such production we = head & Body

Resolves what it generates

↳ nothing derives from them $\rightarrow$ produces 1 symbol

breakfast → protein "with" breakfast **on the slide**
breakfast → protein;
breakfast → bread;
protein → crispiness "crispy" "bacon";
protein → "sausage";
protein → cooked **meat**;
crispiness → "really";
crispiness → "really" crispiness;
cooked → "scrambled";
cooked → **poached**;
cooked → "fried";
bread → "toast";
bread → "biscuits";
bread → **breakfast roll**;

↳ there is no context in the case

With that, every nonterminal in the string has been expanded until it finally contains only terminals and we're left with:

breakfast

protein "with" breakfast "on the side"

cooked "eggs" bread

coached eggs with English muffin on the side

Reproduction

Expressions

- in addition to quoted strings for **terminals that match exact lexemes**, we CAPITALIZE terminals that are a single lexeme whose text representation may vary. NUMBER is any number literal, and STRING is any string literal. Later, we'll do the same for IDENTIFIER.

↳ Animals can watch kermes!!!

```

expression → literal
| unary
| binary
| grouping ;
literal → NUMBER | STRING | "true" | "false" | "nil" ;
grouping → "(" expression ")" ;
unary → "-" expression ;
binary → expression operator expression ;
operator → "=" | "<" | "<=" | ">" | ">=" |
  "<=" | "<=" | "<=" ;

```

Working with Trees
Put on your imagination hat for a moment. Even though we aren't there yet, consider what the interpreter will do with the *syntax* trees. Each kind of expression in *Lox* behaves differently at runtime. That means the interpreter needs to select a different chunk of code to handle each expression type. With tokens, we can simply switch on the *TokenType*. But we don't have a "type" enum for the *syntax* trees, just a separate Java class for each one. We could write a long chain of type tests:

```
We could write a long chain of type tests
if (expr instanceof Expr.Binary) {
    // ...
} else if (expr instanceof Expr.Grouping) {
    // ...
} else {
    // ...
}
```

`src/AsiPrinter.java`

```

lox/AstPrinter.java
create new file
<<
lox/AstPrinter.java
add after print()

```

<<
This recursion is also why people think the Visitor pattern itself has to do with trees.

```
private String parentheses(String name, Expr... exprs) {
    StringBuilder builder = new StringBuilder();
    builder.append("(").append(name);
    for (Expr expr : exprs) {
```

But all of those sequential type tests are slow. Expression types whose names are alphabetically later would take longer to execute because they'd fall through more if cases before finding the right type. That's not my idea of an elegant solution.

We have a family of classes and we need to associate a chunk of behavior with each one. The natural solution in an object-oriented language like Java is to put that behavior into methods on the classes themselves. We could add an abstract `interpret()` method on `Expr` which each subclass then implements to interpret itself.

This works alright for tiny projects, but it scales poorly. Like I noted before, these tree classes span a few domains. At the very least, both the parser and interpreter will mess with them. As you'll see later, we need to do name resolution on them. If our language was statically typed, we'd have a type

If we added instance methods to the expression classes for every one of those operations, that would smush a bunch of different domains together. That violates separation of concerns and leads to hard to maintain code.

```
builder.append("(");
builder.append(expr.accept(this));
}
builder.append(")");
return builder.toString();
}
```

It takes a name and a list of subexpressions and wraps them all up in parentheses, yielding a string like:

```
(+ 1 2)
```

Note that it calls `accept()` on each subexpression and passes in itself. This is the recursive step that lets us print an entire tree.

Parsing

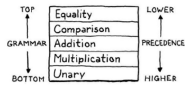
The only remaining piece is parsing—transmogrifying a sequence of tokens into one of those syntax trees

Given a string—a series of tokens—we map those tokens to terminals in the grammar to figure out which rules could have generated that string.

Recursive descent is the simplest way to build a parser, and doesn't require using complex parser generator tools like Yacc, Bison or ANTLR. All you need is straightforward hand-written code. Don't be fooled by its simplicity, though. Recursive descent parsers are fast, robust, and can support sophisticated error-handling. In fact, GCC, V8 (the JavaScript VM in Chrome), Roslyn (the C# compiler written in C#) and many other heavyweight production language implementations use recursive descent. It rocks.

Recursive descent is considered a top-down parser because it starts from the top or outermost grammar rule (here expression) and works its way down into the nested subexpressions before finally reaching the leaves of the syntax tree. This is in contrast with bottom-up parsers like LR that start with primary expressions and compose them into larger and larger chunks of syntax.

Like the scanner, the parser consumes a flat input sequence, only now we're reading tokens instead of characters. Each method for parsing a grammar rule produces a syntax tree for that rule and returns it to the caller.



SYNTAX TREE

- Unary → single operand
- Binary → tree
- Literal → name

```
package com.craftinginterpreters.lex;

abstract class Expr {
    static class Binary extends Expr {
        Binary(Expr left, Token operator, Expr right) {
            this.left = left;
            this.operator = operator;
            this.right = right;
        }
    }

    final Expr left;
    final Token operator;
    final Expr right;
}

// Other expressions...
```

Handwritten notes:

- Red arrows point from `Expr` in `final Expr left;` and `final Expr right;` to the text "Pointer to other elements".
- Red arrows point from `Token operator;` to the text "not" and from `Token` to the text "AG 2".

Class lives in the declaration grammar rule !!

Like we do, we're gonna start with syntax. A `class` statement introduces a new name, so it lives in the declaration grammar rule.

```
declaration → classDecl
            | funDecl
            | varDecl
            | statement ;

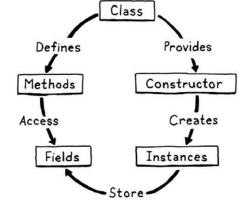
classDecl → "class" IDENTIFIER "(" function* ")" ;
```

The new `classDecl` rule relies on the `function` rule we defined earlier. To refresh your memory:

```
function → IDENTIFIER "(" parameters? ")" block ;
parameters → IDENTIFIER ( "," IDENTIFIER )* ;
```

Handwritten notes:

- Red arrow points from `parameters` to the text "I'll need to define this to parse the".
- Red arrow points from `parameters` to the text "May need to look to escape as well".



In plain English, a class declaration is the class keyword, followed by the class's name, then a curly brace body. Inside that body is a list of method declarations.

Yes, in this current form, you're setting yourself up for a circular dependency problem. Let's break it down simply:

Where the circular dependency happens

You have:

- `Expression.h` (defining `BinaryExpression`, `GroupingExpression`, etc.)
- `IPrettyPrintVisitor.h` (defining the visitor interface)
- `PrettyPrintVisitor.h` (implements the visitor methods using expression classes)

Now, both:

- `Expression` subclasses include (or need to know about) `IPrettyPrintVisitor`, and
- `IPrettyPrintVisitor` needs to know about the concrete expression classes (`BinaryExpression`, etc.)

This creates a cycle: `PrettyPrintVisitor.h` → needs `BinaryExpression` (from `Expression.h`)
`Expression.h` → needs `IPrettyPrintVisitor.h`

How to fix the circular dependency

Use **forward declarations** in `IPrettyPrintVisitor.h`. This avoids including the full definition of each expression class.

Change this:

```
#include "BinaryExpression.h" // or full Expression headers
```

To this:

```
class BinaryExpression;
class GroupingExpression;
class LiteralExpression;
class UnaryExpression;
This tells the compiler:
"These are classes that exist; I'll see their full definitions later."
```

Why it works

The visitor interface only uses **pointers** to the expression classes:

```
virtual std::string visitBinaryExpression(BinaryExpression* expr) = 0;
```

Since it's just using pointers (not the full class), a forward declaration is enough. But in the `.cpp` file where you implement the visitor methods, you **must** include the full class definitions, like this:

```
#include "BinaryExpression.h"
#include "LiteralExpression.h"
// etc.
```

Summary

To avoid circular dependencies:

- In `IPrettyPrintVisitor.h`: use **forward declarations** of expression classes

O erro que você está enfrentando: undefined reference to 'table for BinaryExpression' significa que o compilador encontrou as declarações das classes com métodos virtuais, mas não conseguiu encontrar as implementações dos destrutores virtuais. Como resultado, ele não conseguiu gerar a **virtual table**, necessária para o polimorfismo em tempo de execução.

Solução

Você declarou os destrutores, mas **não os implementou** em lugar nenhum. O linker não consegue encontrá-los.

Por exemplo, no seu `Expressions.h`:

```
class BinaryExpression : public Expression {
public:
    BinaryExpression(Expression* left, Expression* right, const std::string& op);
    ~BinaryExpression(); // <- Você declarou, mas não implementou
};
```

Você precisa **implementar os destrutores** no `Expressions.cpp`:

```
#include "Expressions.h"
// Construtores
BinaryExpression::BinaryExpression(Expression* left, Expression* right, const std::string& op)
: left(left), right(right), op(op) {}
GroupingExpression::GroupingExpression(Expression* expression)
: expression(expression) {}
LiteralExpression::LiteralExpression(const std::string& value)
: value(value) {}
UnaryExpression::UnaryExpression(const std::string& op, Expression* right)
: op(op), right(right) {}
// Destrutores (mesmo que vazios, precisam ser definidos)
BinaryExpression::~BinaryExpression() {}
GroupingExpression::~GroupingExpression() {}
LiteralExpression::~LiteralExpression() {}
UnaryExpression::~UnaryExpression() {}
```

Por que isso acontece?

Toda classe com métodos virtuais (como `accept()`) gera uma **virtual e**, e para isso, **precisa que o destrutor virtual tenha implementação real** (mesmo que vazia). Se o destrutor não for implementado no `.cpp`, o linker reclamará com esse erro porque não consegue montar a **virtual table** da classe.

Checklist para resolver

1. Verifique se há métodos virtuais declarados (como `accept()`)

definitions, like this:
#include "BinaryExpression.h"
#include "LiteralExpression.h"
// etc.

Summary

To avoid circular dependencies:

- In iPrettyPrintVisitor.h use **forward declarations** of expression classes.
 - In .cpp files (like PrettyPrintVisitor.cpp): **include the full headers** of the expressions.
 - Avoid including Expression.h inside iPrettyPrintVisitor.h if not absolutely necessary.
- Let me know if you want me to rewrite the headers using this correct pattern.

Toda classe com métodos virtuais (como accept) gera uma **vtable** e, para isso, **precisa que o destrutor virtual tenha implementação real** (mesmo que vazia).
Se o destrutor não for implementado no .cpp, o linker reclamará com esse erro porque não consegue montar a vtable da classe.

Checklist para resolver

1. Verifique se há métodos virtuais declarados (como accept)
2. Garanta que todos os destrutores virtuais tenham implementação (mesmo vazia!)
3. Coloque essas implementações no .cpp, não no .h

Depois disso, seu projeto deve compilar com sucesso e você poderá seguir com a escrita dos testes unitários, que posso te mostrar em seguida com base no seu Visitor PrettyPrintVisitor.
Quer que eu te mostre um exemplo de teste usando Google Test com uma LiteralExpression e o PrettyPrintVisitor?

Pretty Print

↳ turn into human readable strings

$-(123 * 45.67)$

```
Unary  
(-)  
|  
Binary  
(*)  
/ \  
Literal Grouping  
123 45.67
```

↳ This expression has more expressions inside that knows how to accept a visitor, and the visitor knows how to handle each type

↳ Basically I need to declare each type so I can test

↳ But it would get the pointers in the future !!

```
// Represents: (42 + 3)  
GroupingExpression(  
  new BinaryExpression(  
    new LiteralExpression("42"),  
    new LiteralExpression("3"),  
    "+"  
  )  
)
```

```
// Represents: ((1 + 2))  
Expression* left = new LiteralExpression("1");  
Expression* right = new LiteralExpression("2");  
Expression* bin = new BinaryExpression(left, right, "+");  
Expression* unary = new UnaryExpression(bin);  
Expression* group = new GroupingExpression(unary);
```

```
PrettyPrintVisitor visitor;  
std::string result = group->accept(&visitor);  
EXPECT_EQ(result, "((1 + 2))");
```

↳ There is a recursion here but only a part of it

Absolutely! Let's walk step-by-step through what pretty-printing does, how your Visitor pattern works, and how to write a simple unit test for it in C++.

What Is Pretty Printing?

Pretty printing is just a way to turn an abstract syntax tree (AST) into a human-readable string.

For example, the following expression:

`-(123 * (45.67))`

Would generate this AST:

```
Unary  
(-)  
|  
Binary  
(*)  
/ \  
Literal Grouping  
123 45.67
```

And when pretty-printed using parentheses-style notation (Lisp-style), you'd get:
`(- (* 123 (45.67)))`

How the Visitor Works Here

Each expression (Binary, Grouping, Literal, Unary) knows how to accept a visitor, and the visitor knows how to handle each type.

So:

```
UnaryExpression* expr = new UnaryExpression("-", ...);  
expr->accept(&visitor); // will call visitor.visitUnaryExpression(this)
```

Simple Example (With Manual AST Creation)

Let's write a simple test case that builds this expression:

`-(123 * (45.67))`

1. Include the Headers

Make sure your test includes the AST class headers.

```
#include "PrettyPrintVisitor.h"  
#include "LiteralExpression.h"  
#include "UnaryExpression.h"  
#include "BinaryExpression.h"  
#include "GroupingExpression.h"  
#include <gtest/gtest.h>
```

2. Construct the AST

```
TEST(PrettyPrintVisitorTest, HandlesNestedExpressions) {  
  // Create nodes manually  
  Expression* number123 = new LiteralExpression("123");  
  Expression* number4567 = new LiteralExpression("45.67");  
  Expression* grouping = new GroupingExpression(number123, number4567);  
  Expression* multiply = new BinaryExpression(number123, grouping, "*");  
  Expression* negation = new UnaryExpression("-", multiply);  
  // Visit and assert  
  PrettyPrintVisitor visitor;  
  std::string result = negation->accept(&visitor);  
  EXPECT_EQ(result, "(- (123 * (45.67)))");  
  // Cleanup  
  delete negation; // (assumes destructors delete children)  
}
```

You Might Be Missing This...

Your accept() methods and constructors are **private** — that makes testing impossible.

To fix this:

1. Make Constructors public:

Update each class like this:

```
class BinaryExpression : public Expression {  
public:  
  BinaryExpression(Expression* left, Expression* right, const std::string& op)  
    : left(left), right(right), op(op) {}  
  ...  
};
```

2. Ensure You Cleanup Memory

Unless you're using `std::shared_ptr`, you should implement destructors like this:

```
~BinaryExpression() {  
  delete left;  
  delete right;  
}  
// (Or switch to smart pointers!)
```

What Will Be Printed?

If everything is correct, the AST will produce this string:

`(- (123 * (45.67)))`

But your visitor does:

```
return expr->left->accept(this) + " " + expr->op + " " + expr->right->accept(this);
```

That would produce:

`123 * (45.67)`

So the unary expression will be:

`-123 * (45.67)`

Which is not ideal for clarity. You should wrap **Binary expressions** in parentheses too:

Fix BinaryExpression Pretty Print

Update your method:

```
std::string PrettyPrintVisitor::visitBinaryExpression(BinaryExpression* expr) {  
  return "(" + expr->left->accept(this) + " " + expr->op + " " + expr->right->accept(this) + ")";  
}
```

Now the final output will be:

`(- (123 * (45.67)))`

Perfect!

Summary

1. Make constructors public.
2. Fix visitBinaryExpression to wrap in ().
3. Build a sample AST manually.
4. Use Google Test to write unit tests.
5. Compare the returned string from PrettyPrintVisitor with your expected string.

Let me know if you want a complete main() function or if you want to use `shared_ptr` instead of raw pointers for easier memory management.